

FUNDAMENTALS: ALGORITHMS, Searching Sorting Examples

Dr. Saeed Arif Shah

What is Algorithm? A RECAP

- ▶ Is a complete list of the **steps** (finite and Definite sequence of instructions) necessary to perform a task/computation or for solving a problem.
- ▶ The steps maybe a **general descriptions** (not very detail) or may be totally **precise description** (very detail).
- ▶ Often used in Mathematics, computing, linguistics, and related subjects for calculation and data processing.
- ▶ The term is a corruption of the name al-Khowarizmi (9th century mathematician) – Algorism.
- ▶ Algorism was used for the rules for performing arithmetic using decimal notation.
- ▶ Algorism evolved into the word algorithm by 18th century.

PROBLEM SOLVING WITH ALGORITHMS

Problem Solving with Algorithms

- The Five Phases of Algorithmic Problem Solving
- **Analysis & Specification:** Define the inputs, outputs, and requirements.
- **Design & Representation:** Create the step-by-step logic (the algorithm).
- **Verification & Correction:** Prove the algorithm is logically sound.
- **Analysis of Efficiency:** Determine how fast and scalable the algorithm is.
- **Implementation & Testing:** Translate the algorithm into code and debug it.

Focus: The first four phases are about the **ALGORITHM**; the last phase is about the **CODE**.

• Phase 1: Analysis & Specification

Goal: Define the problem's **scope**, **inputs**, **outputs**, and **constraints** with absolute clarity.

The 3 W's:

- **What** are the required inputs and the desired outputs?
- **What** are the constraints (e.g., memory limits, required speed)?
- **What** are the special/edge cases (e.g., zero, negative numbers, empty data sets)?
- **Strategy/Paradigm :**
 - **Divide and Conquer:** If the problem is too large, break it down into smaller, defined sub-problems.
- **Deliverable:** A clear Problem Specification Document.

Phase 2: Design and Representation

- **Goal:** Translate the defined requirements into a concrete, executable plan (the algorithm).
- **Informal Description (The Idea):**
 - State the solution idea in broad, conversational terms (plain English).
 - Focus on **what** the algorithm must achieve, step-by-step, without worrying about coding syntax.
 - *Example: "First, check the list size. Then, go through each number one by one, keeping a running total."*
- **Formal Representation (The Blueprint):**
 - Use structured tools to express the logic precisely.
 - **Pseudocode:** Use keywords like IF-THEN-ELSE, WHILE, and FOR to detail the sequence and control flow.
 - **Flowcharts:** Use graphical symbols to map the decision points and process order.
- **Strategy Choice:**
- Select the appropriate algorithmic technique (e.g., Greedy, Sorting, Searching) before writing the formal steps.

Phase 3: Verification & Correction

Ensure the algorithm works for *all* valid inputs, correcting any logical flaws before coding.

- **Process: The Walkthrough (or Desk Check):**
- Choose simple, valid input data, including critical edge cases.
- Manually trace the steps of the algorithm, tracking variable changes line by line.
- Confirm that the final output matches the expected result for every test case.
- **Principle:** Logic errors are cheapest to fix here. Verification prevents costly bugs that occur only under rare circumstances.

Phase 4: Analysis of Efficiency (Scaling with Big Data)

Core Concept: We must measure how the algorithm's runtime and memory usage scale as the input size (n) increases.

Tool: Big-O Notation (O): A mathematical tool to describe the worst-case growth rate.

Efficiency	Notation	Description
Excellent	$O(1)$	Constant Time (Instantaneous)
Very Good	$O(\log n)$	Time increases very slowly (e.g., Binary Search)
Good	$O(n)$	Time scales linearly with n
Poor	$O(n^2)$	Time scales quadratically (Avoid for large n)

Describe an algorithm by Natural Languages

EXAMPLE 1: Describe an algorithm for finding the maximum (largest) value in a finite sequence of integers

1. Assume the first integer in the sequence is the largest (temporary maximum).
2. Compare the next integer in the sequence to the temporary maximum. If it is larger than the temporary maximum, set the temporary maximum equal to this integer.
3. Repeat STEP 2 for the next integer in the sequence.
4. Stop when there are no integers left in the sequence. The temporary maximum at this point is the largest integer in the sequence.

Describe an algorithm by Pseudocode

- ❑ Pseudocode is an **informal** programming language
- ❑ Pseudocode provides an **intermediate step** between Language description of an algorithm and an implementation of this algorithm in an Programming Language.
- ❑ A computer program can be produced in any **computer language** using the pseudocode description as a starting point.
- ❑ Advantage: simple, it can be easily written and understood

Describe an algorithm by Pseudocode

Characteristics of Pseudocode used in this class

1. Algorithm Entry Statements – Algorithm maximum (L : list of integers)
 - gives the name of an algorithm
2. Assignment Statements – symbol $:=$ is used; example, ($\text{max} := a$)
OR $\text{max} \leftarrow a$
 - Used to assign values to variables
3. Blocks of Statements
 - Group into blocks using **begin** and **end** statements
4. Comments – ex, { x is the largest element in the list L }
5. Conditional Constructions – **if** condition **then** statement, **else if then**
6. Loop Constructions – **for**, **while**
7. Loops within Loops -
8. Using function in Other functions

Describe an algorithm by Pseudocode

EXAMPLE : Describe an algorithm for finding the maximum (largest) value in a finite sequence of integers

Algorithm $max(a_1, a_2, \dots, a_n: \text{integers})$

$max := a_1$

for $i := 2$ **to** n

if $a_i > max$ **then** $max := a_i$

{ max is the largest element}

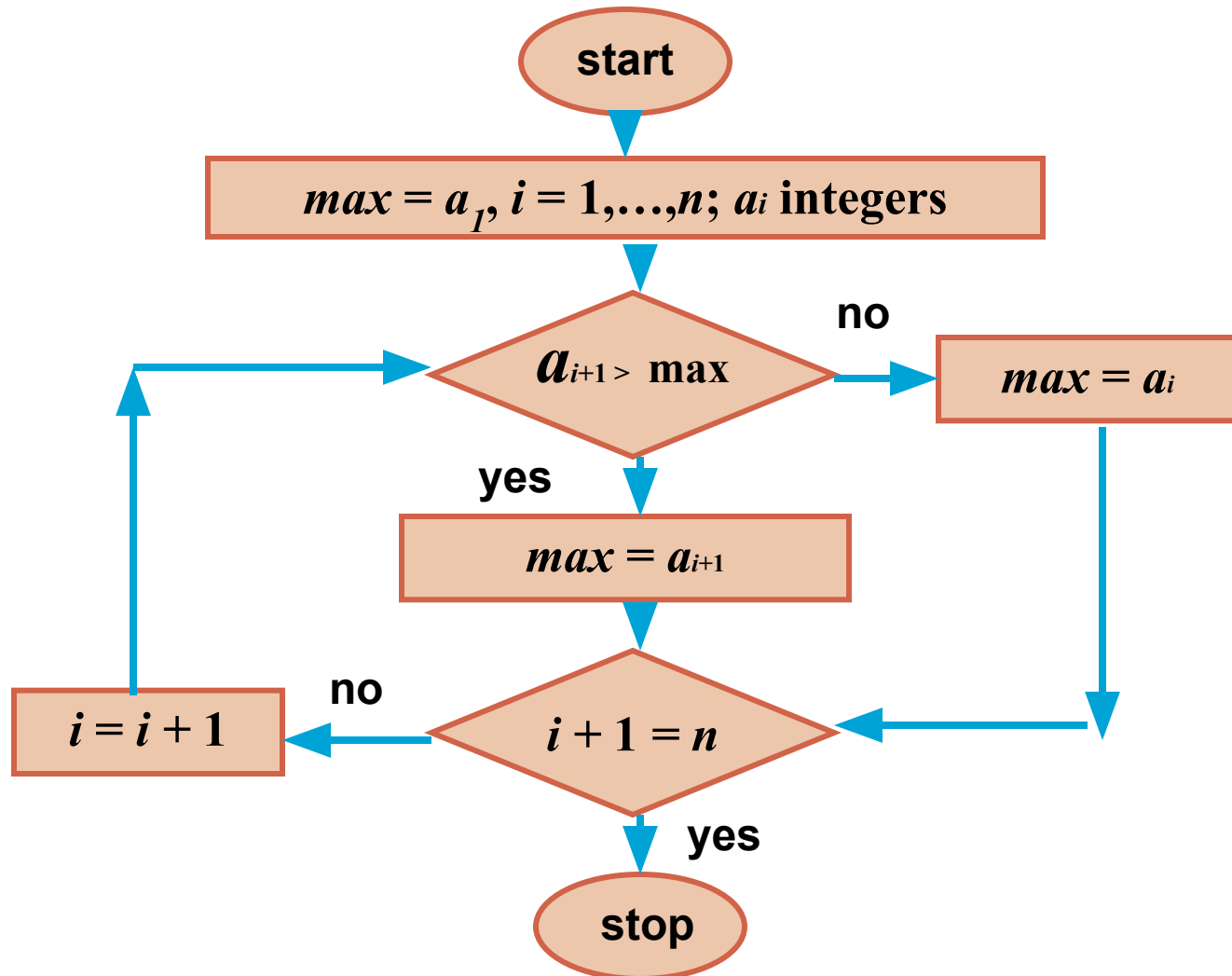
- **Procedure statement**
- **Assignment statement**
- **Loop construction**
- **Conditional construction**
- **Comment**

Describe an algorithm by Flowchart

- A diagrammatic representation of an algorithm

EXAMPLE :

Describe an algorithm for finding the maximum (largest) value in a finite sequence of integers



Describe an algorithm by Programming Languages

- A programming language is a language used to write computer programs
- Some common programming language
 - C, C++, C#, Java, etc

EXAMPLE :

Describe an algorithm for finding the maximum (largest) value in a finite sequence of integers

Write down your own code and test your answer.

EXERCISES

1. Describe an algorithm that takes a list of n integers $a_1, a_2, \dots, a_n$ and finds the number of integers each greater than five in the list.
2. Describe an algorithm that takes a list of integers $a_1, a_2, \dots, a_n$ ($n \geq 2$) and finds the second-largest integer in the sequence.
3. Describe an algorithm that takes a list of n integers ($n \geq 1$) and finds the location of the last even integer in the list, or returns 0 if there are no even integers in the list.
4. Describe an algorithm that takes a list of n integers ($n \geq 1$) and finds the average of the largest and smallest integers in the list.

EXERCISEs

5. What are the problems with following algorithms

a) **Procedure** *double* (*n*: positive integer)

while $n > 0$

$n := 2n$

b) **Procedure** *sum* (*n*: positive integer)

$sum := 0$

while $i < 10$

$sum := sum + i$

SEARCHING ALGORITHM

- Solve problem of locating an element in an ordered lists.
- **Example:** checks the spelling of words in dictionary

GENERAL DESCRIPTION OF SEARCHING ALGORITHM

- Locate an element x in a list of distinct elements $a_1, a_2, \dots, a_n$, or determine that it is not in the list.
- **The solution**
 - The location of the term in the list that equals to x (that is, i is the solution if $x = a_i$) and is 0 if x is not in the list.

SEARCHING ALGORITHM – Linear Search

- Begins by comparing x and a_i
- If $x = a_1$, the solution is the location a_1 , namely 1
- When $x \neq a_1$, compare x with a_2
- If $x = a_2$, the solution is the location a_2 , namely 2
- Continue and repeat the process until a match is found
- If the entire list has been searched without locating x , the solution is 0.

Pseudocode: The Linear (Sequential) Search Algorithm

```
procedure linear search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : distinct  
integers)  
   $i := 1$   
  while ( $i \leq n$  and  $x \neq a_i$ )  
     $i := i + 1$   
  if  $x = a_i$  then  $location := i$   
  else  $location := 0$   
  {location is the subscript of the term that equals  $x$ , or is 0 if  
   $x$  is not found }
```

ANALYSIS OF THE ALGORITHM

- How shall we measure the amount of work done by an algorithm?
- Basic Operation:
 - Comparison of x with an array entry
- Worst-Case Analysis:
 - Let $W(n)$ be a function. $W(n)$ is the maximum number of basic operations performed by the algorithm on any input size n .
 - For our example, clearly $W(n) = n$.
 - The worst cases occur when K appears only in the last position in the array OR when K is not in the array at all.

SORTING ALGORITHM

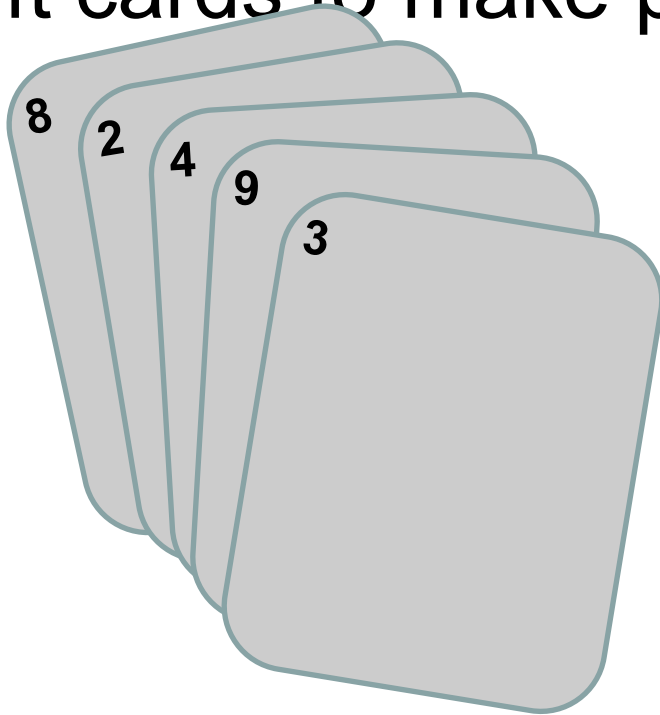
- Solve problem of ordering the elements of a list.
- **Example:** telephone directory, staff directory
- Sorting will put these elements into a lists in which the elements are in increasing orders
 - number: 1, 2, 3,...
 - alphabet: a, b, c,...
- Many algorithm have been developed, but we will discussed only Insertion sort algorithm

Insertion Sort

- Sorting problem
 - Input : unordered data
 - 10 0 4 5 1
 - Output : ordered data
 - 0 1 4 5 10 Ascending order
 - 10 5 4 1 0 Descending order

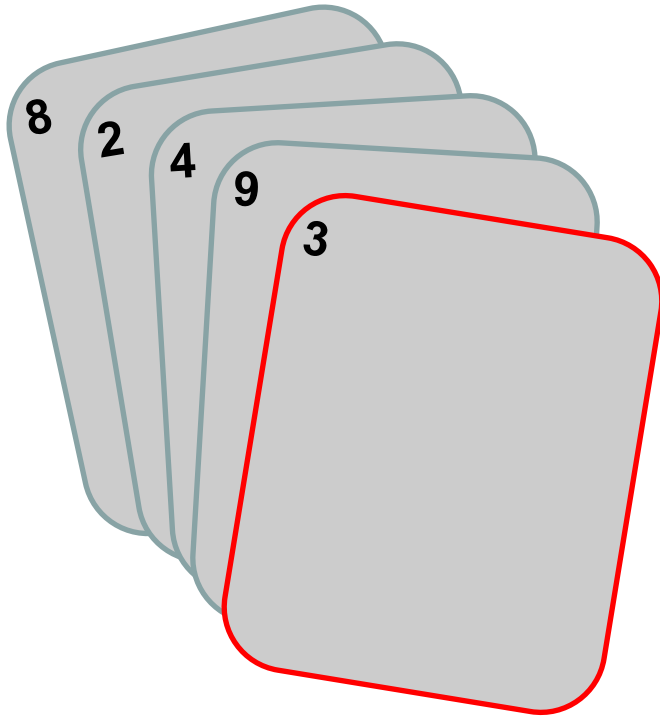
Insertion Sort

- Based on the game of cards
- Basic idea : Starting from the first card shift cards to make place for that card



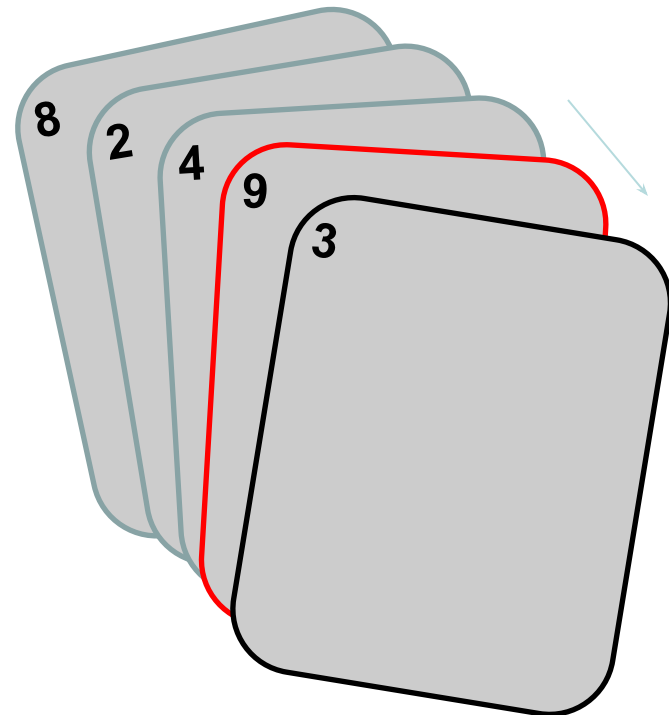
Insertion Sort

- Step 1 Pick Card with value 3
- It is in place



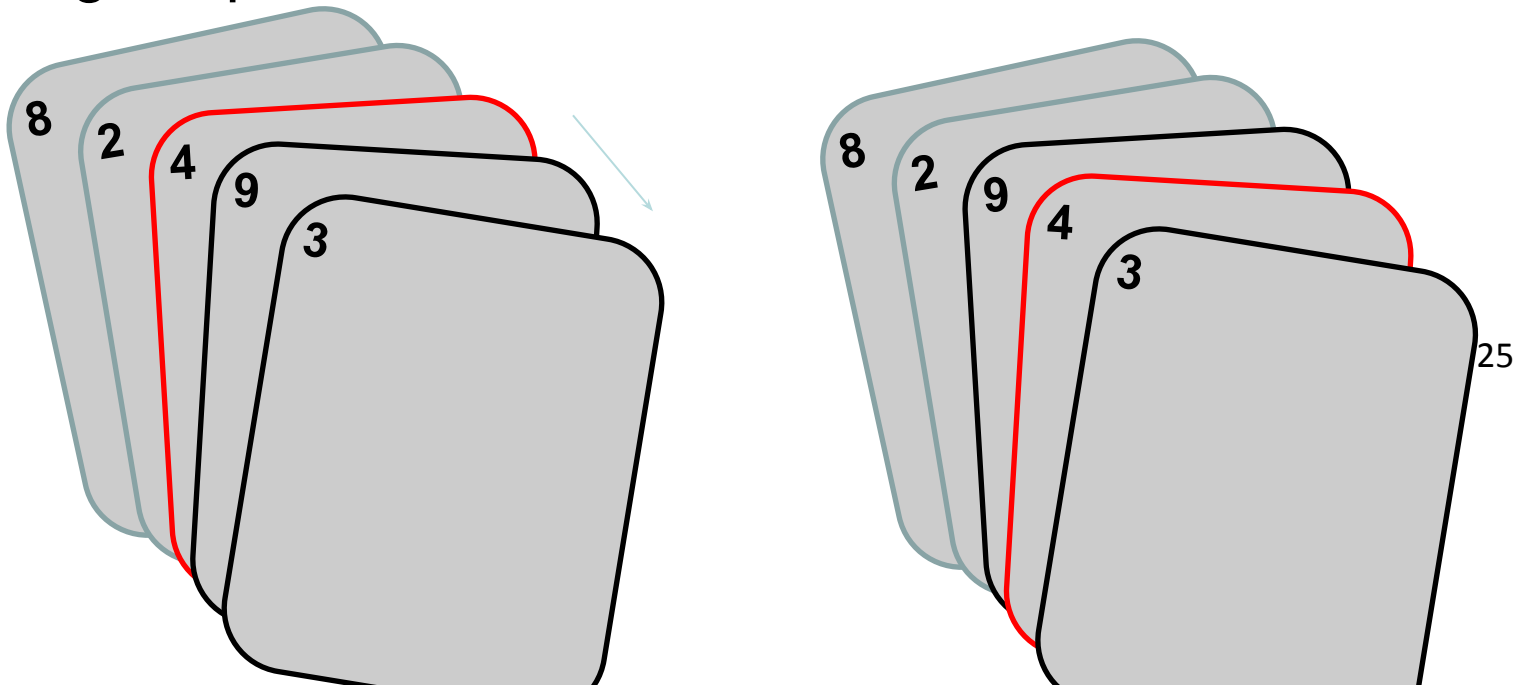
Insertion Sort

- Step 2 Pick card with value 9
- Compare its value to the cards we have seen starting from itself
- No shifting required



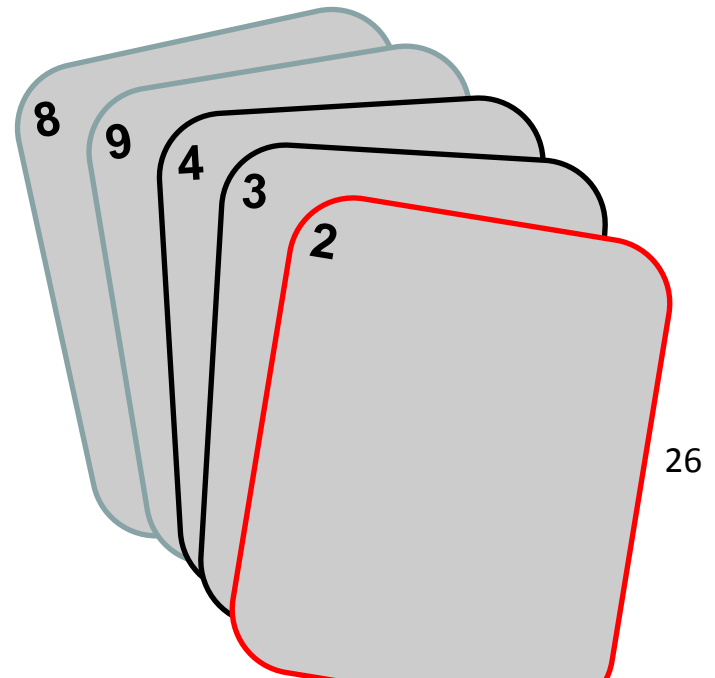
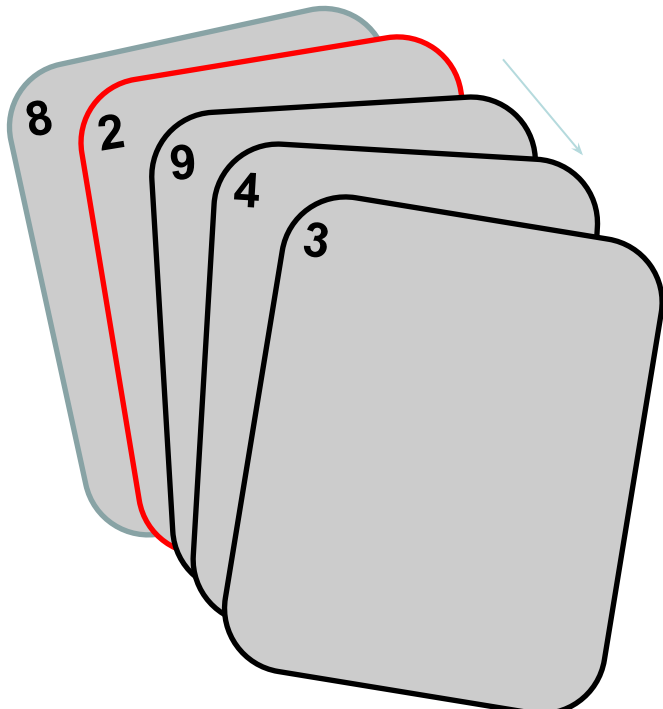
Insertion Sort

- Step 3 Pick card with value 4
- Compare its value to the cards we have seen starting from itself
- Card 9 is out of place No shifting required for card with value 3
- Shift card with value 9 to left & place the current card in the original position of the card 9



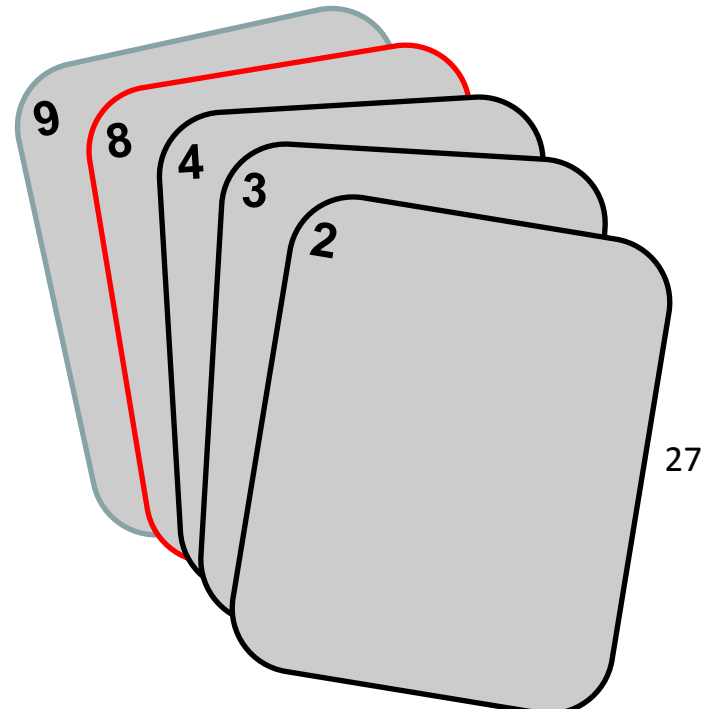
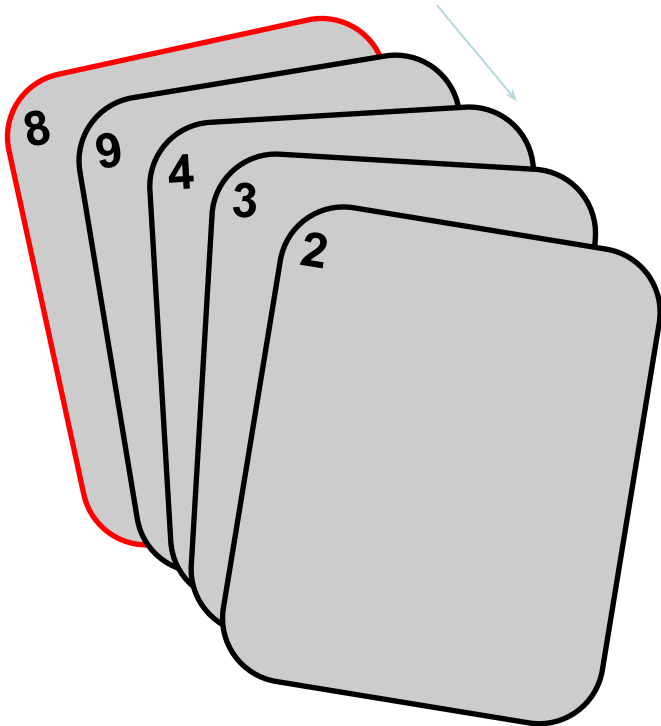
Insertion Sort

- Step 4 Pick card with value 2
- Compare its value to the cards we have seen starting from itself
- All the cards seen so far require shifting
- Shift all the cards that we have seen so far to left and place the card with value 2 at the top



Insertion Sort

- Step 5 Pick card with value 8
- Compare its value to the cards we have seen starting from itself
- 9 is out of order 2, 3 and 4 are in order
- Card with value 9 is out of order shift it to left and place the current card in its place



Insertion Sort

- Some subtle points
 - We don't need to check the first card with itself
- Algorithm
 - Pick the current card
 - Locate its position in the required sorted order
 - Shift other cards to make room for the current card
 - Place the current card in the position determined in the step 2

Insertion Sort

- Algorithm

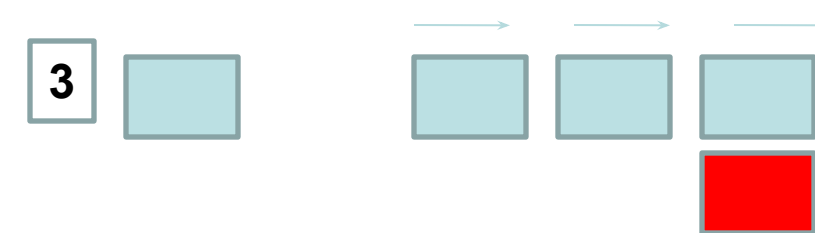
1. Pick the current card



2. Locate its position in the required sorted order



3. Shift other cards to make room for the current card



4. Place the current card in the position determined in the step 2



Insertion Sort

- Algorithm

1. Pick the current card
2. Locate its position in the required sorted order
3. Shift other cards to make room for the current card
4. Place the current card in the position determined in the step 2

Input Array $A[1 \dots n]$

1. $\text{Key} = A[j]$
2. $i = j - 1$
While ($i > 0$ and $A[i] > \text{key}$)
3. $A[i+1] = A[i]$
 $i = i - 1$
End of while
4. $A[i+1] = \text{key}$

Insertion Sort

- Final Algorithm
- Input $A[1 \dots n]$
- Insertion Sort
 - For $j=2$ to n
 - $\text{Key} = A[j]$
 - $i = j - 1$
 - While($i > 0$ and $A[i] > \text{key}$)
 - $A[i+1] = A[i]$
 - $i = i - 1$
 - End of while
 - $A[i+1] = \text{key}$

Analysis of Algorithms

- Problems in Analysis
 - Different Computing Architectures
 - Different Computer Speeds
- Solution
 - Generalization
 - Models for Algorithmic Analysis

Good Algorithm

- Efficiency
 - Computer Cycles usage
 - Memory usage

Focus of this course

- Main
 - Efficiency in terms of computer time consumption
 - OR the time taken to solve the problem
 - Correctness
- Secondary
 - Memory used

Quote of the Day

"An algorithm must be seen to be believed." — *Donald Knuth*

(You must follow the steps and verify it, not just trust it.)."